

REPORT CONTENT SHEET

Raw material for your futuristic report — Build It in English

Hoja de contenido: materia prima del informe futurista

Student / Estudiante	Cristian Navas
Project / Proyecto	Signal Desk
Pitch	A real-time monitoring system to efficiently prioritize and manage operational incidents and alerts.
Environment / Entorno	MAUI SDK .NET 10
Report medium / Medio del informe	Word exported to PDF
Date / Fecha	2026-09-04
Teacher / Docente	Ing. Javier Arturo García Mineros

Reminder: this sheet is not the report. Transform this content into your futuristic PDF, following the design brief.

Esta hoja no es el informe: transforme este contenido en su PDF futurista siguiendo el brief de diseño.

2. The environment used

Cross-Platform Efficiency (.NET MAUI): Enables the development of native applications for iOS, Android, and desktop platforms from a single codebase using C# and XAML. This dramatically reduces development time, simplifies code maintenance, and takes advantage of the mature .NET ecosystem without sacrificing the native performance of the devices.

Backend Agility (Firebase): By using a Backend-as-a-Service (BaaS) model, Firebase eliminates the need to configure, maintain, and scale physical or virtual servers. Integrated services such as Firestore (real-time database) and Firebase Authentication allow complex features to be implemented quickly, significantly reducing the project's time-to-market.

Eficiencia Multiplataforma (.NET MAUI): Permite compilar aplicaciones nativas para iOS, Android y escritorio desde un único código base utilizando C# y XAML. Esto reduce drásticamente el tiempo de desarrollo, facilita el mantenimiento del código y aprovecha el ecosistema maduro de .NET sin

sacrificar el rendimiento nativo de los dispositivos.

Agilidad en el Backend (Firebase): Al utilizar un modelo de Backend-as-a-Service (BaaS), Firebase elimina la necesidad de configurar, mantener y escalar servidores físicos o virtuales. Servicios integrados como Firestore (base de datos en tiempo real) y Firebase Authentication permiten implementar funciones complejas de forma rápida, reduciendo significativamente el time-to-market del proyecto.

2.1 Alternative not used

Flutter (Dart)

Why: Although Flutter is a highly capable cross-platform framework, it requires learning and adopting Dart. .NET MAUI was selected instead to leverage the team's existing strong proficiency in C# and to integrate seamlessly with standard .NET libraries, avoiding a steep learning curve and accelerating initial development.

Flutter (Dart)

Por qué: Aunque Flutter es un framework multiplataforma muy capaz, requiere aprender y adoptar el lenguaje Dart. En su lugar, se seleccionó .NET MAUI para aprovechar la sólida experiencia que ya tiene el equipo con C# y para lograr una integración perfecta con las bibliotecas estándar de .NET. Esto evita una curva de aprendizaje pronunciada y acelera el desarrollo inicial.

3. Tools

TOOL	VERSION	WHAT IT WAS USED FOR (ENGLISH)
.NET MAUI	10.0.100	Our cross-platform framework for the creation and development of the system using C# and XAML.
Blazor Bootstrap	4.0.0 Stable Version	To implement pre-built interfaces without the need to write code manually.
Firebase Console (SQL Connect)	17.3.2	Database Administrator

4. Resources

RESOURCE TYPE	DETAIL	CONTRIBUTION TO THE PROJECT
Hardware	Development laptop Windows and smartphone (Android/iOS) for testing.	Main equipment used to write the codebase, compile the application for different platforms, and evaluate native performance.
Time invested	Total estimated 3 hours and 17 min	Development effort required to complete the application lifecycle, from design to bug fixing.
Documentation / tutorials	Official Microsoft Learn documentation for .NET MAUI	Provide the reference architecture, best practices, and code examples to implement the UI
Libraries or packages	Specific NuGet packages such as Plugin.Firebase, FirebaseAuthentication.net, and CommunityToolkit.Maui.	Reduce development time by incorporating authentication features, data synchronization, and visual components without programming them from scratch.
AI or assistance used	Large Language Models like Gemini, ChatGPT, and Claude	Speed up bug resolution, assist in designing XAML views, and help structure complex Firebase database queries.

5. Functionality

FEATURE	WHAT THE USER DOES AND SEES (ENGLISH)
1- Ticket Login	The user fills in the title, description, and selects a priority level (P1-P4). Upon clicking "Log Ticket", a new entry appears in the main list with auto-calculated SLA due dates
2- Status Workflow	The user selects a ticket from the dashboard and clicks "Mark as Resolved" or updates its state. The status tag updates dynamically from New/InProgress to Resolved
3- Priority Filtering	The user selects a priority filter option on the dashboard. The list updates instantly to display only tickets matching the selected priority level

4- Data Persistence	The user closes and reopens the application. All previously logged tickets and resolution logs remain intact and load automatically from local JSON storage

5.1 Out of scope

The application does not support multi-user role management or cloud backend synchronization. It does not send email or SMS notifications for overdue SLAs, nor does it integrate with third-party ticketing tools like Jira or ServiceNow

La aplicación no admite la gestión de roles para múltiples usuarios ni la sincronización con un backend en la nube. Tampoco envía notificaciones por correo electrónico o SMS cuando se vencen los SLA, ni se integra con herramientas de gestión de tickets de terceros como Jira o ServiceNow.

6. Architecture and data model

The application follows the MVVM (Model-View-ViewModel) architectural pattern in .NET MAUI. The Models folder contains data structures (Ticket.cs). The ViewModels folder handles presentation logic and commands (MainViewModel.cs). The Views folder contains XAML pages (MainPage.xaml) bound to the ViewModels via Data Binding

*La aplicación utiliza el patrón arquitectónico **MVVM (Modelo-Vista-ViewModel)** en .NET MAUI. La carpeta **Models** contiene las estructuras de datos (**Ticket.cs**). La carpeta **ViewModels** se encarga de la lógica de presentación y los comandos (**MainViewModel.cs**). La carpeta **Views** contiene las páginas en XAML (**MainPage.xaml**), vinculadas a los ViewModels mediante **Data Binding**.*

6.1 Data persistence

ata resides in-memory as an ObservableCollection<Ticket> while the application is running to ensure fast UI updates. For persistence, ticket data is serialized and stored locally on the disk in JSON format (tickets.json)

*Los datos se almacenan en memoria mediante una **ObservableCollection<Ticket>** mientras la aplicación está en ejecución, lo que permite actualizaciones rápidas de la interfaz de usuario. Para*

la persistencia, los datos de los tickets se serializan y se almacenan localmente en el disco en formato ****JSON (tickets.json)****.

Insert here the diagram of your data model.
Inserte aquí el diagrama de su modelo de datos.

7. Code highlights

FILE AND METHOD	WHAT IT DOES	WHY IT IS INTERESTING
Ticket.cs – Model	Handles the presentation logic and application actions.	Applies the MVVM pattern, separating the application logic from the user interface.
MainViewModel.cs – Commands	Keeps tickets in memory and automatically updates the interface.	Allows changes to the data to be reflected quickly in the UI.
ObservableCollection<Ticket>	Connects visual elements with data and ViewModel commands.	Reduces the need to write manual code to control the interface.

Paste the three code excerpts below, with their English comments.
Pegue los tres fragmentos de código con sus comentarios en inglés.

8. Test log

ID	WHAT WAS TESTED	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
T-01	Ticket creation	Verify that ticket information is registered correctly.	The ticket is created and appears in the list.	The ticket is displayed correctly in the list.	Passed
T-02	Ticket display	Verify that tickets are loaded correctly in the interface.	Registered tickets appear correctly.	Tickets are displayed correctly in the interface.	Passed

T-03	Ticket update	Verify that changes to a ticket are reflected in the application.	The modified information is updated correctly.	The changes are reflected in the list.	Passed
T-04	Ticket deletion	Verify the deletion of a selected ticket.	selected ticket. The selected ticket is removed from the list.	The ticket is removed correctly.	Passed
T-05	UI updates	Verify the functionality of ObservableCollection<Ticket>.	The interface updates automatically when data changes.	The list updates correctly.	Passed
T-06	Saving to tickets.json	Verify data persistence.	Tickets are correctly stored in the JSON file.	Data is successfully saved in JSON format.	Passed
T-07	Loading stored data	Verify that saved tickets can be retrieved.	Stored tickets appear again when the application starts.	Saved data is retrieved correctly.	Passed
T-08	XAML interface and navigation	interface and navigation Verify that pages and controls work correctly.	The interface allows the main functions to be used without errors.	Navigation and controls work correctly.	Passed

9. Challenges and conclusions

PROBLEM	WHAT I SEARCHED IN ENGLISH	HOW I SOLVED IT
Need to mock a database before connecting the real backend.	"net maui mock data in memory observablecollection"	I created a service class (MockDataService) with ObservableCollection lists to temporarily store and manage data in memory while the app is running.
In-memory data was resetting when navigating between certain pages.	"net maui keep state singleton service dependency injection"	I registered my mock data service as a Singleton in MauiProgram.cs to maintain a single, persistent state throughout the entire application session.

Simulating the response time of a real database to test loading indicators in the UI.	c# simulate async network call task delay"	I added await Task.Delay(1000) to the asynchronous methods in my in-memory service to simulate network latency and verify the app's visual behavior.
---	--	--

9.1 Conclusions

In this initial phase of the project, we focused heavily on building a robust cross-platform frontend using .NET MAUI and the MVVM architecture. Because the Firebase backend is not yet fully implemented, we relied on an in-memory data service to mock database interactions. This approach allowed us to rapidly prototype the UI, test data binding, and perfect navigation without being blocked by backend configuration. The main lesson learned is the importance of coding against interfaces (e.g., *IDataStore*); this way, replacing our current mock service with the real Firebase service in the future will require minimal changes to our ViewModels.

*En esta fase inicial del proyecto, nos enfocamos en construir un frontend multiplataforma robusto usando .NET MAUI y la arquitectura MVVM. Debido a que el backend de Firebase aún no está implementado, utilizamos un servicio de datos en memoria para simular las interacciones con la base de datos. Este enfoque nos permitió crear un prototipo rápido de la interfaz, probar el enlace de datos y perfeccionar la navegación sin depender de la configuración del servidor. El principal aprendizaje fue la importancia de programar utilizando interfaces (ej. *IDataStore*); de esta manera, reemplazar nuestro servicio simulado actual con el servicio real de Firebase en el futuro requerirá cambios mínimos en nuestros ViewModels.*

Declaration / Declaración

I declare that I wrote the code and the report myself, that every identifier and comment is in English, and that I can explain any part of this work on request.

Declaro que escribí el código y el informe, que todo identificador y comentario está en inglés y que puedo explicar cualquier parte de este trabajo.

Cristian Navas
Student / Estudiante

Ing. Javier A. García Mineros
Teacher / Docente